

Ejercicio de Programación Orientada a Objetos

1. Descripción del Proyecto: Convertidor Multiespecie de Edad Canina y Felina

El Problema

Se requiere desarrollar un sistema que calcule la edad biológica de una mascota (perro o gato) en relación con los años humanos. No es una conversión lineal simple; la lógica debe considerar que el desarrollo de un animal cambia según su especie y, en el caso de los perros, según su tamaño (Pequeño, Mediano o Grande).

El Reto Técnico (Arquitectura)

El desarrollo se divide en dos componentes estrictamente separados:

Capa de Lógica (logica/calculadora.py): Contiene las clases y fórmulas matemáticas. No puede usar `print()` ni `input()`.

Capa de Vista (vista/consola_vista.py): Gestiona la interacción con el usuario, captura datos y muestra los resultados calculados por la lógica.

Estructura de la Capa de Lógica

Se debe implementar una jerarquía de clases donde la lógica sea "pura".

Clase Padre: Mascota

Es la base general para cualquier animal.

Atributos: `nombre` (str) y `edad_cronologica` (int).

Método `__init__`: Inicializa los datos básicos de la mascota.

Método `calcular_edad_humana(self)`: Un método base que, por defecto, podría retornar la edad multiplicada por un factor estándar, pero que será sobrescrito por las clases hijas (Polimorfismo).

Clases Hijas (Especialización)

Clase Gato: El primer año equivale a 15 años humanos, el segundo a 9 más, y luego cada año adicional suma 4 años humanos.

Clase Perro: Debe recibir un atributo adicional llamado tamaño (Pequeño, Mediano, Grande).

Perro Pequeño: Multiplica los años por un factor de 5.

Perro Grande: Multiplica los años por un factor de 7 (debido a su envejecimiento más acelerado).

Estructura de la Capa de Vista

Es la única autorizada para interactuar con el usuario mediante consola.

Método registrar_mascota(): * Muestra un menú: 1. Perro, 2. Gato.

Solicita el nombre, la edad actual y, si es perro, su tamaño.

Retorno: Devuelve una instancia (objeto) de Perro o Gato según la elección.

Método mostrar_resultado():

Llama al método de la lógica calcular_edad_humana().

Imprime el resultado final: *"Tu mascota [Nombre] tiene [X] años humanos"*.

Flujo de Ejecución (main.py)

El archivo raíz actúa como el disparador. Su única responsabilidad es importar la función de inicio de la vista y ejecutarla bajo la condición `if __name__ == "__main__":`.

Reglas de Validación (Criterios de Evaluación)

Para que el ejercicio sea exitoso, el aprendiz debe cumplir con:

Encapsulamiento: La lógica solo procesa números y strings; no imprime nada.

Uso de Herencia: Las clases Perro y Gato deben heredar de Mascota y usar `super().__init__` para el nombre y la edad.

Comunicación por Retorno: La vista nunca calcula la edad manualmente; solo muestra lo que la lógica le devuelve.

2. Ejercicio de POO: Sistema de Gestión de Salud (IMC Pro)

Descripción del Proyecto

Se requiere un sistema para centros médicos que permita registrar pacientes, calcular su Índice de Masa Corporal (IMC) y mantener sus datos actualizados en una lista global.

Estructura de la Capa de Lógica (`logica/entidades.py`)

Se debe implementar una lógica pura, basada en objetos, que gestione la información sin interactuar con la consola.

Clase Persona:

Atributos: nombre (str), apellido (str), edad (int), peso (float), altura (float).

Método `__init__`: Inicializa todos los atributos de la persona.

Método `calcular_imc(self)`: Retorna el valor numérico del IMC

Método `obtener_estado(self)`: Retorna un string según el resultado del IMC:

< 18.5: "Bajo peso"

18.5 - 24.9: "Normal"

25.0 - 29.9: "Sobrepeso"

> 30.0: "Obesidad"

Estructura de la Capa de Gestión (logica/gestor_pacientes.py)

Esta clase se encarga de administrar el **arreglo** de personas.

Atributos: lista_pacientes (una lista vacía al inicio).

Método agregar_paciente(self, persona): Recibe un objeto de la clase Persona y lo guarda en el arreglo.

Método buscar_paciente(self, apellido): Busca en el arreglo y retorna el objeto si coincide con el apellido.

Método actualizar_datos(self, apellido, nuevo_peso, nueva_altura): Localiza a la persona y modifica sus atributos de salud.

Estructura de la Capa de Vista (vista/consola.py)

Única capa autorizada para usar input() y print().

Menú Principal:

Registrar Paciente.

Ver Lista de Pacientes (Muestra nombre, apellido, peso, estatura e IMC y Estado).

Actualizar Peso/Altura (Busca por apellido).

Salir.

Reglas de Validación (Criterios de Evaluación)

Para que el ejercicio sea exitoso, se debe cumplir con:

Encapsulamiento Estricto: La clase Persona no imprime el resultado; retorna el valor para que la Vista lo muestre.

Manipulación de Colecciones: El aprendiz debe demostrar que sabe iterar (for o while) sobre el arreglo para buscar o actualizar datos.

Integridad de Objetos: Al actualizar el peso, el método calcular_imc() debe reflejar el cambio automáticamente la próxima vez que se llame (reactividad del objeto).

Punto de Entrada: El programa debe iniciar desde un archivo main.py usando el bloque `if __name__ == "__main__":`.

3. Ejercicio de POO: Sistema de Gestión de Agenda Médica (Versión Profesional)

Descripción del Proyecto

Se requiere desarrollar un sistema que permita administrar las citas médicas de una clínica. El software debe organizar la información mediante la interacción de tres entidades principales: Paciente, Médico y Cita, gestionadas por un controlador central.

Estructura de la Capa de Lógica (logica/entidades.py)

Se debe implementar una jerarquía de objetos donde la lógica sea "pura", sin interacción con el usuario.

Clase Médico:

Atributos: nombre_medico (str), especialidad (str) e id_registro_medico (str).

Método __init__: Inicializa los datos del profesional.

Clase Cita:

Atributos: fecha (str), hora (str) y objeto_medico (instancia de la clase Médico).

Nota: Aquí se aplica **Agregación**: una Cita "tiene un" Médico.

Clase Paciente:

Atributos: nombre_paciente (str), id_paciente (str) y lista_citas (list).

Método agregar_cita(self, nueva_cita): * **Validación:** Verifica que el paciente no tenga más de 3 citas programadas.

Retorno: Devuelve un valor booleano o mensaje de éxito/error.

Estructura de la Capa de Negocio (logica/gestor_clinica.py)

Esta clase actúa como el cerebro del sistema y maneja la persistencia en memoria.

Clase GestorClinica:

Atributo: diccionario_pacientes (Llave: id_paciente, Valor: Objeto de tipo Paciente).

Método programar_cita(self, id_paciente, objeto_cita): Busca al paciente en el diccionario y delega la responsabilidad de agregar la cita al objeto Paciente.

Estructura de la Capa de Vista (vista/consola_vista.py)

Es la única autorizada para interactuar con el usuario mediante consola.

Método registrar_cita():

Solicita datos del paciente (si no existe, lo crea).

Solicita datos del médico y crea el objeto Médico.

Crea el objeto Cita integrando al médico.

Envía todo al GestorClinica.

Método mostrar_agenda(): Imprime todas las citas de un paciente, detallando el nombre del médico y su especialidad.

Reglas de Validación (Criterios de Evaluación)

Para que el ejercicio sea exitoso, el aprendiz debe cumplir con:

Comunicación entre Objetos: El objeto Cita debe recibir un objeto Médico como parámetro, no solo su nombre.

Encapsulamiento: Ninguna clase de la lógica puede usar print().

Uso de Diccionarios y Listas: Se debe evidenciar el almacenamiento de objetos Paciente en un diccionario y de objetos Cita en una lista interna del paciente.

Punto de Entrada: El archivo main.py debe ser el único disparador del programa.

4. Ejercicio de POO: Motor de Blackjack "Las Vegas Edition"

Descripción del Proyecto

Desarrollar un simulador de cartas que gestione una baraja de 52 naipes (poker). El sistema debe ser capaz de representar visualmente las cartas con sus pintas reales, calcular puntajes dinámicos y aplicar las reglas de las figuras (J, Q, K) y el As (A).

Estructura de la Capa de Lógica (logica/modelos.py)

Clase Carta:

Atributos: pinta (str: , , , ) y rango (str: 2-10, J, Q, K, A).

Método __str__: Debe retornar la combinación de ambos, por ejemplo: "10 o "A.

Método obtener_valor(): * Si el rango es numérico, retorna su valor entero.

Si es J, Q o K, retorna 10.

Si es A, retorna 11 (valor inicial).

Clase Mano:

Atributo: cartas (lista de objetos Carta).

Método calcular_total():

Suma los valores de todas las cartas en la lista.

Lógica del As: Mientras el total sea mayor a 21 y exista al menos un "A" en la mano, resta 10 puntos al total (convirtiendo ese As de 11 a 1).

Clase Baraja:

Atributo: mazo (lista generada automáticamente con las 4 pintas y los 13 rangos).

Método mezclar(): Utiliza la librería random para desordenar el mazo.

Método dar_carta(): Retorna un objeto Carta y lo elimina del mazo.

Estructura de la Capa de Vista (vista/consola.py)

Es la responsable de la "puesta en escena":

Interfaz de Juego: Muestra las cartas del jugador usando los símbolos: "Tus cartas: [8♣] [A♦] - Total: 19".

Ciclo de Decisión: Pregunta al usuario si desea pedir otra carta o plantarse.

Reglas de Validación (Criterios de Evaluación)

El aprendiz debe demostrar dominio en:

Encapsulamiento: La lógica solo entrega objetos y números; la vista es la que decide cómo imprimirlos con los símbolos.

Uso de `super()` o Constructores: Si decides que `ManoJugador` y `ManoDealer` hereden de una clase base `Mano`.

Manejo de Colecciones: Uso correcto de `.pop()` o `.append()` para mover objetos `Carta` entre la `Baraja` y la `Mano`.

Lógica Algorítmica: El ajuste automático del valor del As según el contexto de la mano.

5. Reto de Ingeniería: Simulación de Juego de Azar "Estado Compartido"

1. El Problema

Se requiere desarrollar un sistema que simule el juego de mesa conocido como **"¿Por qué siempre yo?"**.



El objetivo es que se traduzcan las reglas físicas a un modelo digital utilizando Programación Orientada a Objetos (POO).

2. Reglas del Juego (Contexto)

- **Componentes:** Un tablero con 6 huecos (numerados del 1 al 6), un dado de 6 caras y fichas (canicas).
- **Mecánica:**
 1. Cada jugador inicia con un número equitativo de fichas (ej. 10 fichas cada uno).
 2. En su turno, el jugador lanza el dado.
 3. Huecos 1 al 5: * Si el hueco está vacío, el jugador coloca una de sus fichas allí.
 - Si el hueco ya tiene una ficha, el jugador debe recogerla y sumarla a sus fichas personales (aquí es donde el jugador pierde porque se llena de fichas).
 4. **Hueco 6:** Este hueco es un agujero profundo. La ficha que caiga aquí desaparece del juego para siempre. El jugador simplemente se deshace de una ficha.
- **Condición de Victoria:** Gana el primer jugador que se quede **sin** fichas.

3. El Reto de Análisis

A diferencia de ejercicios anteriores, **no se proporcionará la arquitectura**. Debes realizar el siguiente proceso:

- **Identificación de Entidades:** ¿Qué elementos del mundo real deben convertirse en clases? (Mínimo 3 entidades).
- **Definición de Atributos:** ¿Qué datos necesita conocer cada objeto para funcionar?
- **Lógica de Interacción:** ¿Qué objeto es responsable de lanzar el dado? ¿Quién decide si una ficha se pone o se quita del tablero?
- **Arquitectura de Capas:** Debes mantener la separación estricta:

- **Capa de Lógica:** Solo reglas y cálculos (sin input ni print).
- **Capa de Vista:** Gestión de turnos y visualización de lo que ocurre en el tablero.

4. Criterios de Evaluación

Para que el ejercicio sea exitoso, se debe haber resuelto por su cuenta:

- **El estado del Tablero:** ¿Cómo representó los huecos 1 al 5? (¿Un arreglo de booleanos? ¿Una lista de objetos?).
- **El flujo de fichas:** ¿La lógica detecta correctamente cuándo un jugador "limpia" un hueco y cuándo "ocupa" uno?
- **El "Pozo" del 6:** ¿El sistema diferencia que el hueco 6 nunca se llena y siempre resta fichas al jugador?
- **La comunicación:** ¿La vista muestra claramente quién lanzó, qué número cayó y cuántas fichas le quedan a cada uno?